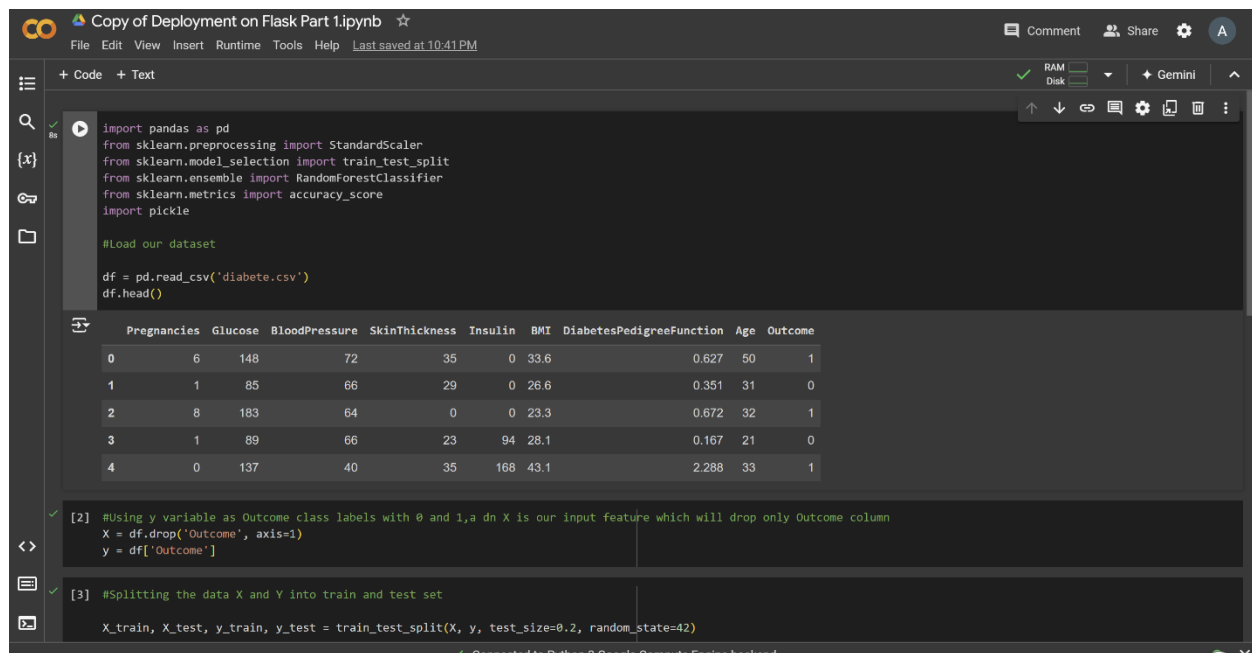


Name: Anar Mammadli

Submission Date: October 27th, 2024

Submitted to: An204o/Datasets

Deployment of Flask (Part 1) Code:



```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import pickle

#Load our dataset

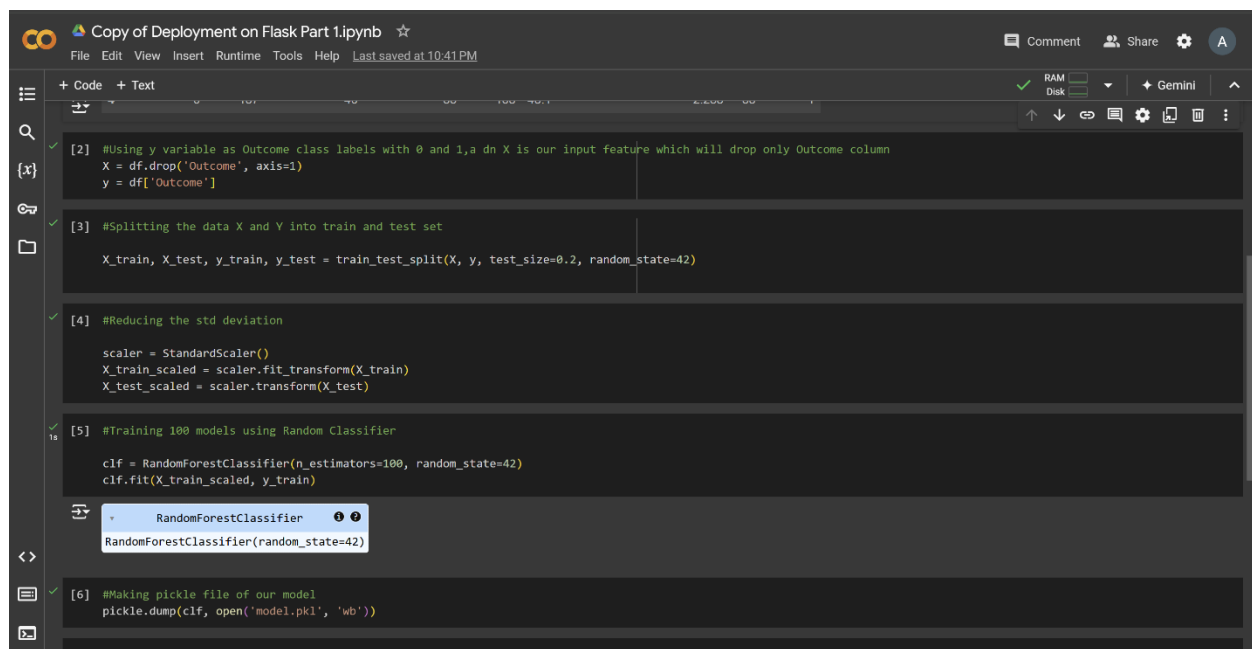
df = pd.read_csv('diabete.csv')
df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

```
[2] #Using y variable as Outcome class labels with 0 and 1,a dn X is our input feature which will drop only Outcome column
X = df.drop('Outcome', axis=1)
y = df['Outcome']

[3] #Splitting the data X and Y into train and test set

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```



```
[2] #Using y variable as Outcome class labels with 0 and 1,a dn X is our input feature which will drop only Outcome column
X = df.drop('Outcome', axis=1)
y = df['Outcome']

[3] #Splitting the data X and Y into train and test set

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

[4] #Reducing the std deviation

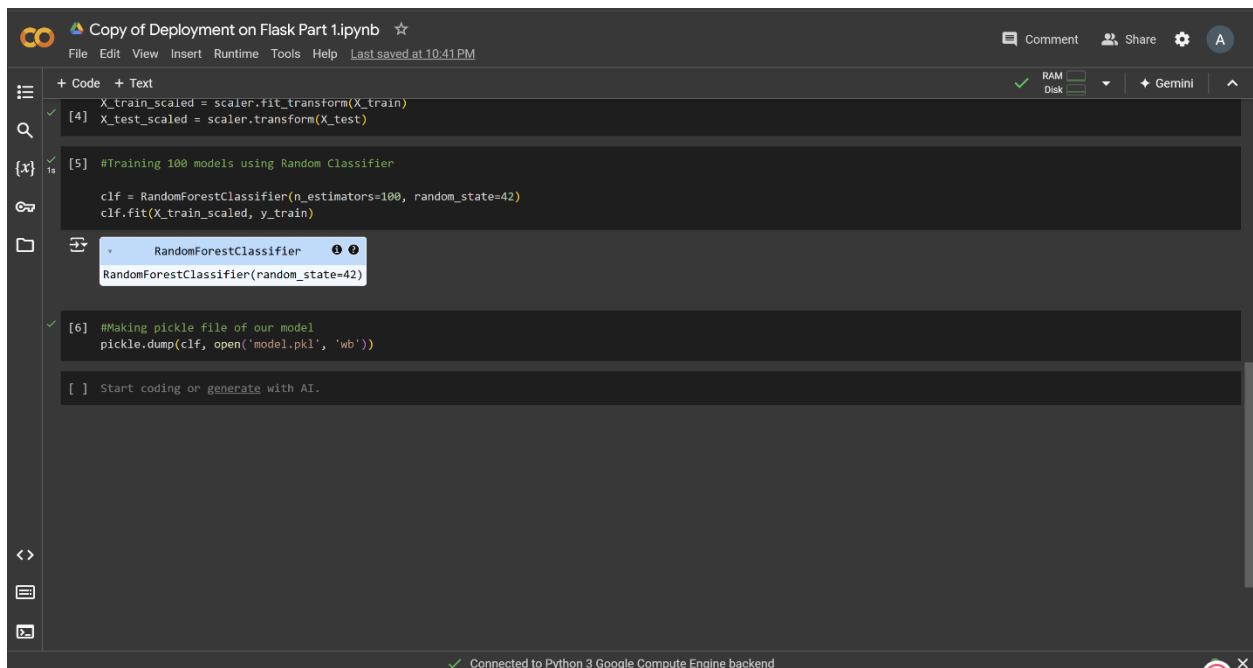
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

[5] #Training 100 models using Random Classifier

clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train_scaled, y_train)

RandomForestClassifier
RandomForestClassifier(random_state=42)

[6] #Making pickle file of our model
pickle.dump(clf, open('model.pkl', 'wb'))
```



Copy of Deployment on Flask Part 1.ipynb

```
[4] X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

[5] #Training 100 models using Random Classifier

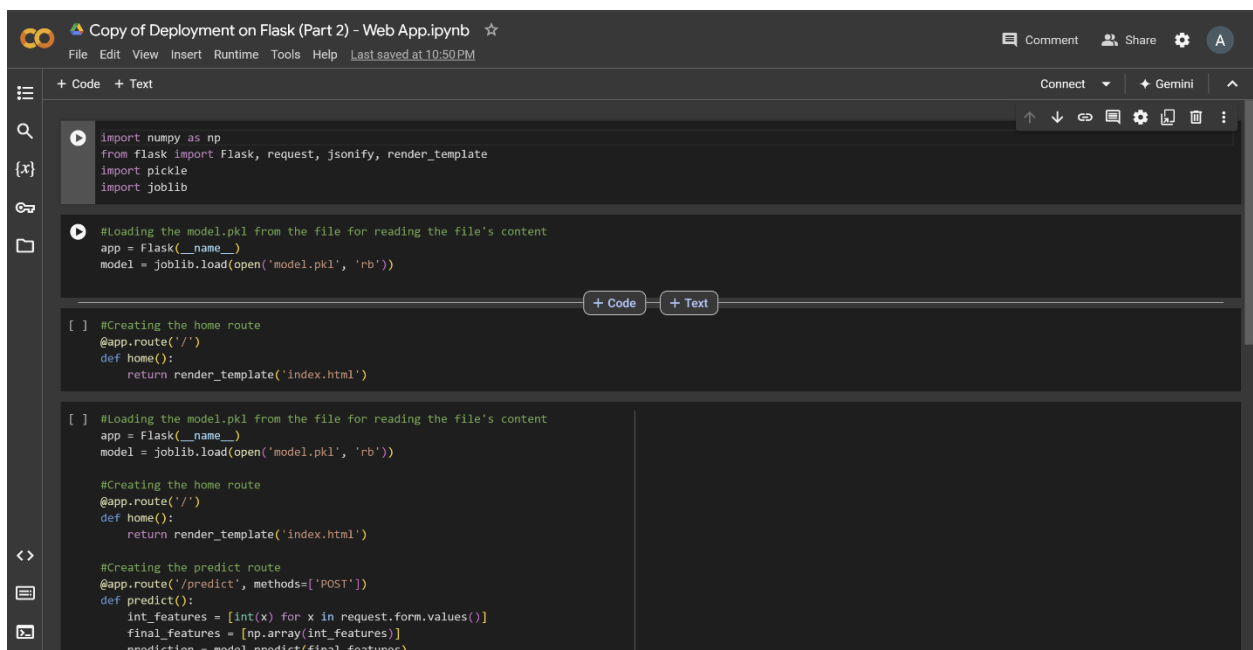
    clf = RandomForestClassifier(n_estimators=100, random_state=42)
    clf.fit(X_train_scaled, y_train)

[6] #Making pickle file of our model
    pickle.dump(clf, open('model.pkl', 'wb'))

[ ] Start coding or generate with AI.
```

Connected to Python 3 Google Compute Engine backend

Flask App Code (Part 2):



Copy of Deployment on Flask (Part 2) - Web App.ipynb

```
import numpy as np
from flask import Flask, request, jsonify, render_template
import pickle
import joblib

#Loading the model.pkl from the file for reading the file's content
app = Flask(__name__)
model = joblib.load(open('model.pkl', 'rb'))

#Creating the home route
@app.route('/')
def home():
    return render_template('index.html')

#Loading the model.pkl from the file for reading the file's content
app = Flask(__name__)
model = joblib.load(open('model.pkl', 'rb'))

#Creating the home route
@app.route('/')
def home():
    return render_template('index.html')

#Creating the predict route
@app.route('/predict', methods=['POST'])
def predict():
    int_features = [int(x) for x in request.form.values()]
    final_features = np.array(int_features)
    prediction = model.predict(final_features)
```

Copy of Deployment on Flask (Part 2) - Web App.ipynb ☆

File Edit View Insert Runtime Tools Help Last saved at 10:50 PM

+ Code + Text

Connect Gemini

```
[ ] #Loading the model.pkl from the file for reading the file's content
app = Flask(__name__)
model = joblib.load(open('model.pkl', 'rb'))

#Creating the home route
@app.route('/')
def home():
    return render_template('index.html')

#Creating the predict route
@app.route('/predict', methods=['POST'])
def predict():
    int_features = [int(x) for x in request.form.values()]
    final_features = [np.array(int_features)]
    prediction = model.predict(final_features)

    output = round(prediction[0], 2)

    return render_template('index.html', prediction_text='Employee Salary should be $ {}'.format(output))

[ ] if __name__ == "__main__":
    app.run(debug=True)
```

* Serving Flask app '__main__'

* Debug mode: on

INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

* Running on <http://127.0.0.1:5000>

INFO:werkzeug:Press CTRL+C to quit

INFO:werkzeug: * Restarting with stat